

Name-Kashish Chelwani

PRN-24070126082

AIML-B1

EXPERIMENT-5 Banking Application

Design a banking application with the following class definitions: Customer, Account, LoanAccount(extends Account), and SavingsAccount(extends Account). Define the methods deposit(), withdraw() in the base class Account and override them in the child classes as required.

In the main class, create arraylist of customers, accounts and display consolidated information of each customer's accounts.

INPUT-

Account.java

```
//Class representing a bank account
public class Account {

    //Unique account number
    protected int accountNumber;

    //Current balance in the account
    protected double balance;

    //Customer associated with this account
    protected customer customer;

    //Constructor to initialize account details
    public Account(int accountNumber, double balance, customer customer) {
        this.accountNumber = accountNumber;
        this.balance = balance;
        this.customer = customer;
    }

    //Method to deposit money into the account
    public void deposit(double amount) {
        balance += amount; // Add amount to balance
        System.out.println("Amount Deposited: " + amount);
        System.out.println("Balance: " + balance);
    }
}
```

```

    }

    //Method to withdraw money from the account
    //Throws InsufficientBalanceException if balance is not enough
    public void withdraw(double amount) throws InsufficientBalanceException {

        //Check if withdrawal amount exceeds current balance
        if(amount > balance)
            throw new InsufficientBalanceException("Insufficient Balance");

        balance -= amount; // Deduct amount from balance
        System.out.println("Amount Withdrawn: " + amount);
        System.out.println("Balance: " + balance);
    }

    //Method to display account and customer details
    public void displayAccount() {
        customer.displayCustomer(); // Display customer details
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Balance: " + balance);
    }
}

```

CurrentAccount.java

```

//Class representing a Current Account (inherits from Account)
public class CurrentAccount extends Account {

    //Minimum balance required for a current account
    double minimumBalance = 1000;

    //Constructor to initialize current account details
    public CurrentAccount(int accountNumber, double balance, customer
customer) {
        // Call parent class constructor
        super(accountNumber, balance, customer);
    }

    //Method to check whether minimum balance is maintained
    public void checkMinimumBalance() {

        //Check if current balance is below minimum required balance
        if(balance < minimumBalance)
            System.out.println("Minimum balance not maintained");
        else
            System.out.println("Minimum balance maintained");
    }

    //Overriding withdraw method specific to CurrentAccount
    @Override

```

```

public void withdraw(double amount) throws InsufficientBalanceException {

    //Check if withdrawal amount exceeds available balance
    if(amount > balance)
        throw new InsufficientBalanceException("Insufficient Balance");

    //Deduct amount from balance
    balance -= amount;

    //Display withdrawal message specific to current account
    System.out.println("Current Account Withdrawal: " + amount);
}
}

```

Customer.java

```

//Class representing a customer
public class customer {

    public String customerID;
    public String firstName;
    public String lastName;
    public String email;
    public String phoneNumber;
    public String address;
    protected String aadharID; //sensitive info
    protected String panID; //sensitive info

    //Constructor to initialize details
    public customer(String customerID, String firstName, String lastName,
                    String email, String phoneNumber, String address,
                    String aadharID, String panID) {

        this.customerID = customerID;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.phoneNumber = phoneNumber;
        this.address = address;
        this.aadharID = aadharID;
        this.panID = panID;
    }

    //Displays customer information
    public void displayCustomer() {
        System.out.println("Customer ID: " + customerID);
        System.out.println("Name: " + firstName + " " + lastName);
        System.out.println("Email: " + email);
    }
}

```

```

        System.out.println("Phone: " + phoneNumber);
        System.out.println("Address: " + address);
        System.out.println("Aadhar ID: " + aadharID);
        System.out.println("PAN ID: " + panID);
    }
}

```

SavingsAccount.java

```

//Class representing a Savings Account
public class SavingsAccount extends Account {

    double interestRate = 0.04; //interest rate

    //Constructor to initialize savings account
    public SavingsAccount(int accountNumber, double balance, customer
customer) {
        super(accountNumber, balance, customer);
    }

    //Calculates and displays interest earned
    public void calculateInterest() {
        double interest = balance * interestRate;
        System.out.println("Interest Earned: " + interest);
    }

    //Withdraw with minimum balance restriction
    @Override
    public void withdraw(double amount) throws InsufficientBalanceException {

        //Ensure minimum balance of 500 is maintained
        if(balance - amount < 500)
            throw new InsufficientBalanceException("Minimum balance 500
required");

        balance -= amount;
        System.out.println("Savings Withdrawal: " + amount);
    }
}

```

Loan.java

```

//Class representing a Loan
public class Loan {

    double loanAmount;
    int months;
}

```

```

double emi; //monthly installment

//Constructor to initialize loan and calculate EMI
public Loan(double loanAmount, int months) {

    this.loanAmount = loanAmount;
    this.months = months;
    this.emi = loanAmount / months; //simple EMI calc
}

//Pays EMI from given account
public void payEMI(Account acc) throws InsufficientBalanceException {

    //Check if account has enough balance
    if(acc.balance < emi)
        throw new InsufficientBalanceException("Not enough balance to pay
EMI");

    acc.balance -= emi;

    System.out.println("EMI Paid: " + emi);
    System.out.println("Remaining Balance: " + acc.balance);
}

//Displays loan details
public void displayLoan() {
    System.out.println("Loan Amount: " + loanAmount);
    System.out.println("Monthly EMI: " + emi);
}
}

```

InsufficientBalanceException.java

```

//Custom exception for insufficient balance
public class InsufficientBalanceException extends Exception {

    //Constructor passing message to parent
    public InsufficientBalanceException(String msg) {
        super(msg);
    }
}

```

InvalidAccountException.java

```

//Custom exception for invalid account
public class InvalidAccountException extends Exception {

```

```

        //Constructor passing message to parent
        public InvalidAccountException(String msg) {
            super(msg);
        }
    }
}

```

Main.java

```

import java.util.ArrayList;

//Main class
public class Main {

    public static void main(String[] args) {

        try {
            //List of customers
            ArrayList<customer> customers = new ArrayList<>();

            //Creating customers
            customer c1 = new customer(
                "082", "Kashish", "Chelwani",
                "kashish811@gmail.com", "9309813833",
                "Nagpur", "99999999999", "ABCDEFGHIJK"
            );

            customer c2 = new customer(
                "091", "Akansha", "Choudhary",
                "Akansha.Choudhary@gmail.com", "8237664844",
                "Mumbai", "Choudhary9", "XYZABCDEFGH"
            );

            customer c3 = new customer(
                "088", "Nayra", "Chelwani",
                "nayra.chelwani@gmail.com", "9960695130",
                "Delhi", "11111111111", "KKKKKKKKKKKK"
            );

            customers.add(c1);
            customers.add(c2);
            customers.add(c3);

            //List of accounts
            ArrayList<Account> accounts = new ArrayList<>();

            SavingsAccount sa1 = new SavingsAccount(9001, 5000, c1);
            CurrentAccount ca1 = new CurrentAccount(8001, 10000, c1);

```

```

SavingsAccount sa2 = new SavingsAccount(5002, 8000, c2);
CurrentAccount ca2 = new CurrentAccount(3002, 15000, c2);

SavingsAccount sa3 = new SavingsAccount(3001, 3000, c3);

accounts.add(sa1);
accounts.add(ca1);
accounts.add(sa2);
accounts.add(ca2);
accounts.add(sa3);

//Display summary
displayInfo(customers, accounts);

// ----- EXCEPTIONS PART -----
System.out.println("\nPerforming Transactions\n");

try {
    sa1.deposit(2000);    //5000 → 7000
    sa1.withdraw(1000);  //7000 → 6000
    sa1.calculateInterest(); //interest ~ 240

    System.out.println();

    //Large Loan to trigger EMI exception
    Loan loan = new Loan(5552000, 12);
    loan.displayLoan();
    loan.payEMI(sa1); //Exception 1

} catch (InsufficientBalanceException e) {
    System.out.println("Exception: " + e.getMessage());
}

//Second exception block
System.out.println("\nPerforming Transactions\n");

try {
    sa1.withdraw(6000); //will break min balance rule → Exception

} catch (InsufficientBalanceException e) {
    System.out.println("Exception: " + e.getMessage());
}

} catch (Exception e) {
    System.out.println("Exception: " + e.getMessage());
}
}

```

```

//Display customers with accounts
public static void displayInfo(ArrayList<customer> customers,
ArrayList<Account> accounts) {

    System.out.println("Customer and Account summary:\n");

    for (customer cust : customers) {

        System.out.println("---- CUSTOMER DETAILS ----");
        System.out.println("Customer ID: " + cust.customerID);
        System.out.println("Name: " + cust.firstName + " " +
cust.lastName);
        System.out.println("Email: " + cust.email);
        System.out.println("Phone: " + cust.phoneNumber);
        System.out.println("Address: " + cust.address);
        System.out.println("Aadhar ID: " + cust.aadharID);
        System.out.println("PAN ID: " + cust.panID);

        System.out.println("\n---- ACCOUNTS ----");

        double totalBalance = 0;
        int accountCount = 0;

        for (Account acc : accounts) {
            if (acc.customer.customerID.equals(cust.customerID)) {

                System.out.println("Account Number: " +
acc.accountNumber);
                System.out.println("Account Type: " +
acc.getClass().getSimpleName());
                System.out.println("Balance: Rs. " + acc.balance);
                System.out.println();

                totalBalance += acc.balance;
                accountCount++;
            }
        }

        System.out.println("Total Accounts: " + accountCount);
        System.out.println("Total Balance Across All Accounts: Rs. " +
totalBalance);
        System.out.println();
    }
}
}

```


OUTPUT-

```
des.java-extension-pack-jdk\java\latest\bin\java.exe --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp C:\Users\Kashish\AppData\Local\Temp\vscodesws_58733\jdt_ws\jdt.ls-java-project\bin Main "
```

Customer and Account summary:

---- CUSTOMER DETAILS ----

Customer ID: 082

Name: Kashish Chelwani

Email: kashish811@gmail.com

Phone: 9309813833

Address: Nagpur

Aadhar ID: 99999999999

PAN ID: ABCDEFGHIJK

---- ACCOUNTS ----

Account Number: 9001

Account Type: SavingsAccount

Balance: Rs. 5000.0

Account Number: 8001

Account Type: CurrentAccount

Balance: Rs. 10000.0

Total Accounts: 2

Total Balance Across All Accounts: Rs. 15000.0

---- CUSTOMER DETAILS ----

Customer ID: 091

Name: Akansha Choudhary

Email: Akansha.Choudhary@gmail.com

Phone: 8237664844

Address: Mumbai

Aadhar ID: Choudhary9

PAN ID: XYZABCDEFGH

---- ACCOUNTS ----

Account Number: 5002

Account Type: SavingsAccount

Balance: Rs. 8000.0

Account Number: 3002

```

---- CUSTOMER DETAILS ----
Customer ID: 091
Name: Akansha Choudhary
Email: Akansha.Choudhary@gmail.com
Phone: 8237664844
Address: Mumbai
Aadhar ID: Choudhary9
PAN ID: XYZABCDEFGH

---- ACCOUNTS ----
Account Number: 5002
Account Type: SavingsAccount
Balance: Rs. 8000.0

Account Number: 3002
Account Type: CurrentAccount
Balance: Rs. 15000.0

Total Accounts: 2
Total Balance Across All Accounts: Rs. 23000.0

---- CUSTOMER DETAILS ----
Customer ID: 088
Name: Nayra Chelwani
Email: nayra.chelwani@gmail.com
Phone: 9960695130
Address: Delhi
Aadhar ID: 11111111111
PAN ID: KKKKKKKKKK

---- ACCOUNTS ----
Account Number: 3001
Account Type: SavingsAccount
Balance: Rs. 3000.0

Total Accounts: 1
Total Balance Across All Accounts: Rs. 3000.0

```

Exceptions

```

Performing Transactions

Amount Deposited: 2000.0
Balance: 7000.0
Savings Withdrawal: 1000.0
Interest Earned: 240.0

Loan Amount: 5552000.0
Monthly EMI: 462666.6666666667
Exception: Not enough balance to pay EMI

Performing Transactions

Exception: Minimum balance 500 required

```

GITHUB LINK-

<https://github.com/Kashish811/Programming-in-Java/tree/main/Banking%20Application>